

Tensor Compression for Quantum Physics: Modern Scientific Software with AI Agents

Hiroshi Shinaoka

October 1, 2026

Saitama University



Slides (PDF)

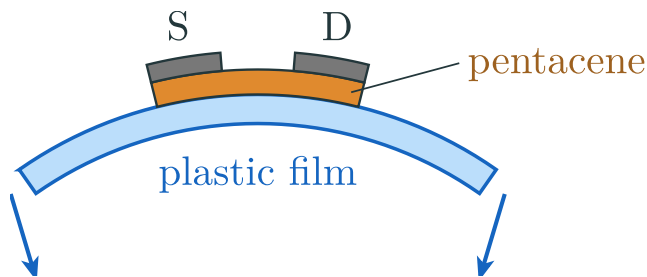
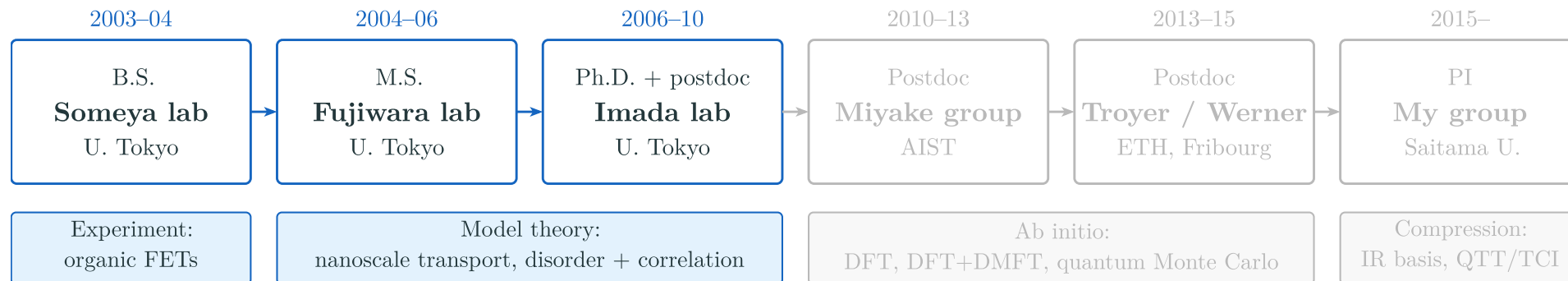
- 5th edition, held in Okinawa
- Feb 28 to Mar 4, 2027
- Okinawa Seinen Kaikan, Naha
- **Open to industry participants**



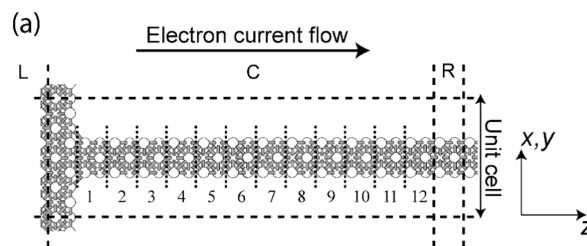
compphysschool.github.io/2027/



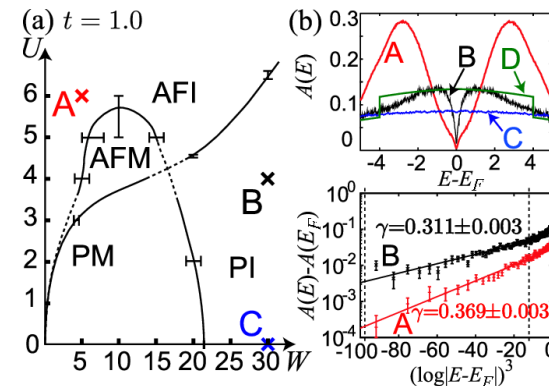
March 2026, Okinawa: 370 participants, 11 sponsoring companies



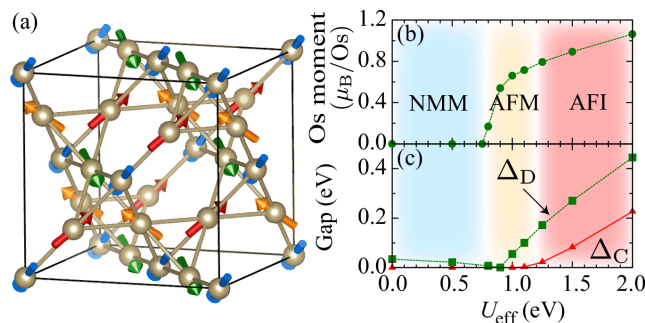
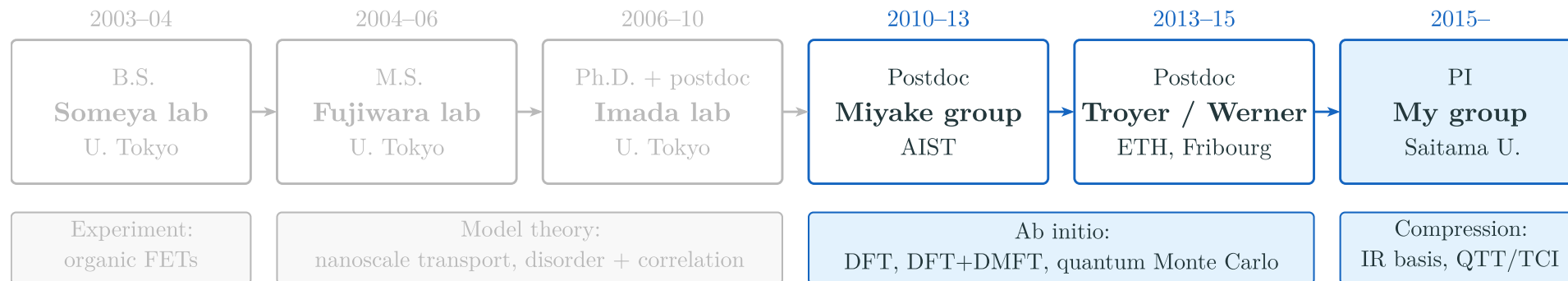
Flexible organic FETs
Sekitani et al., APL **86**, 073511 (2005)



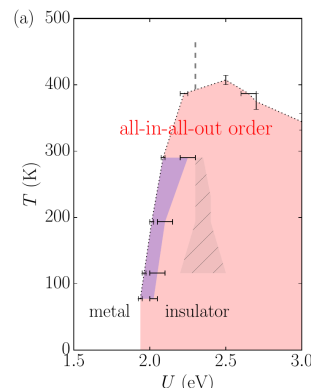
Metal nanowire transport
HS, Hoshi, Fujiwara, JPSJ **77**, 114712 (2008)



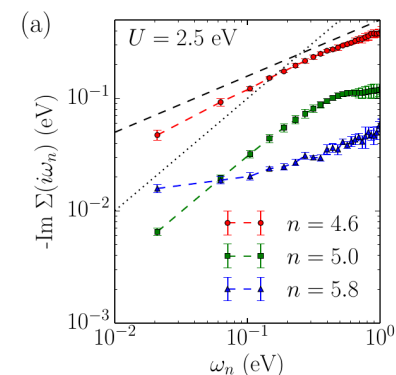
Soft Hubbard gap
HS, Imada, PRL **102**, 016404 (2009)



DFT: 5d pyrochlore $\text{Cd}_2\text{Os}_2\text{O}_7$
 HS, Miyake, Ishibashi, PRL **108**, 247204 (2012)



DFT+DMFT: pyrochlore iridates
 HS, Hoshino et al., PRL **115**, 156401 (2015)



QMC data on $i\omega_n \rightarrow \text{IR basis}$
 HS et al., PRB **96**, 035147 (2017)

Equilibrium

$$G(i\omega_n, \mathbf{k})$$

- $i\omega_n$: Matsubara frequency
- $\mathbf{k}$: momentum

Nonequilibrium

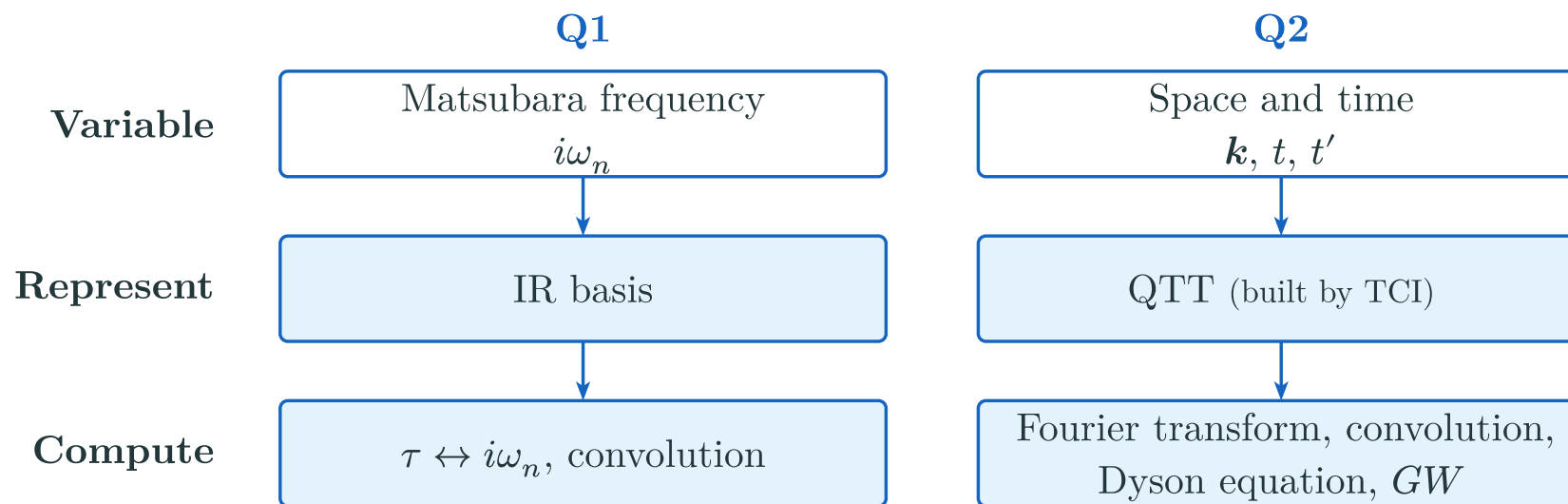
$$G(\mathbf{k}, t, t')$$

- t, t' : two times
- $\mathbf{k}$: momentum

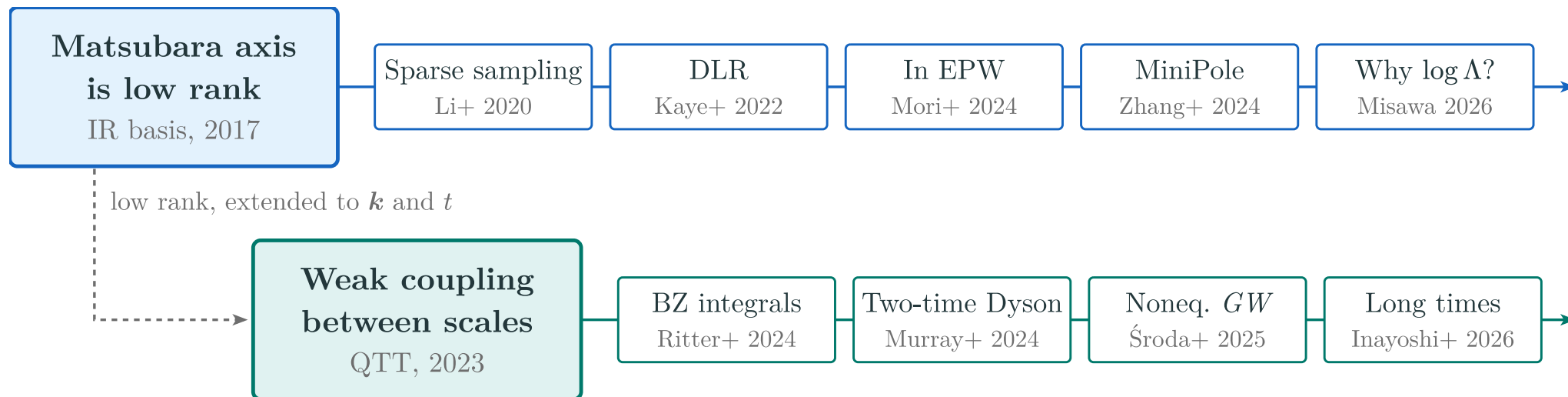
Number of grid points grows with

- lower temperature (denser Matsubara frequencies)
- finer $\mathbf{k}$ mesh
- longer times

Example: 10^4 frequencies $\times 100^3$ $\mathbf{k}$ -points $\sim 10^{10}$; 10^3 times per axis with the same $\mathbf{k}$ mesh $\sim 10^{12}$.



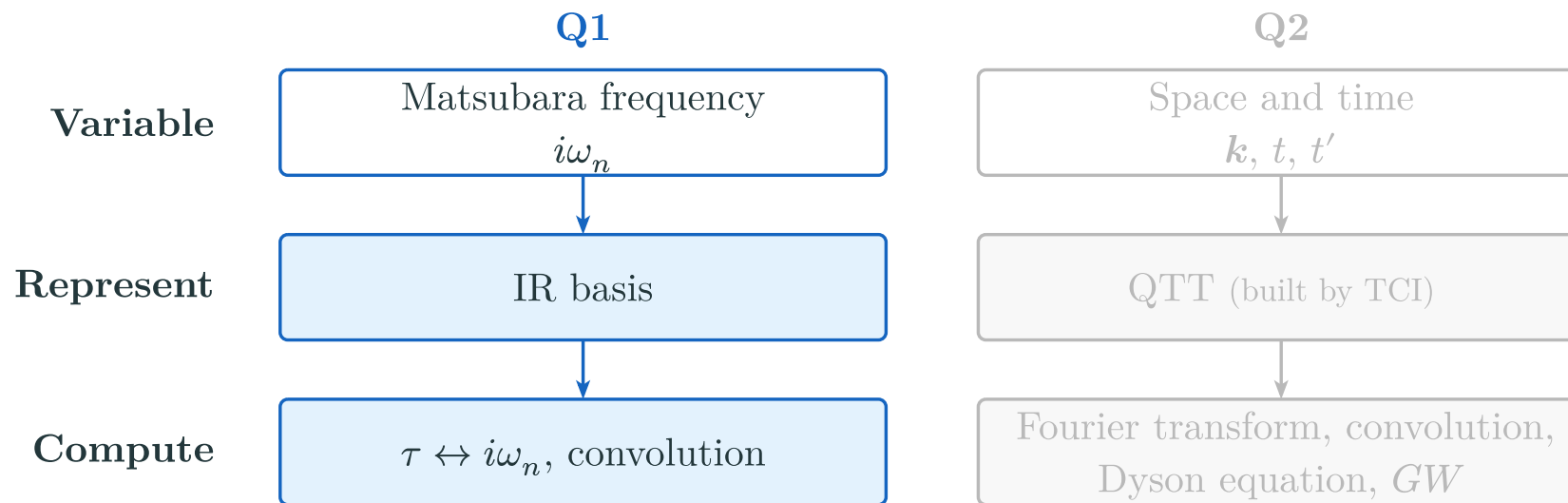
Not only data compression: we compute on the compressed data.



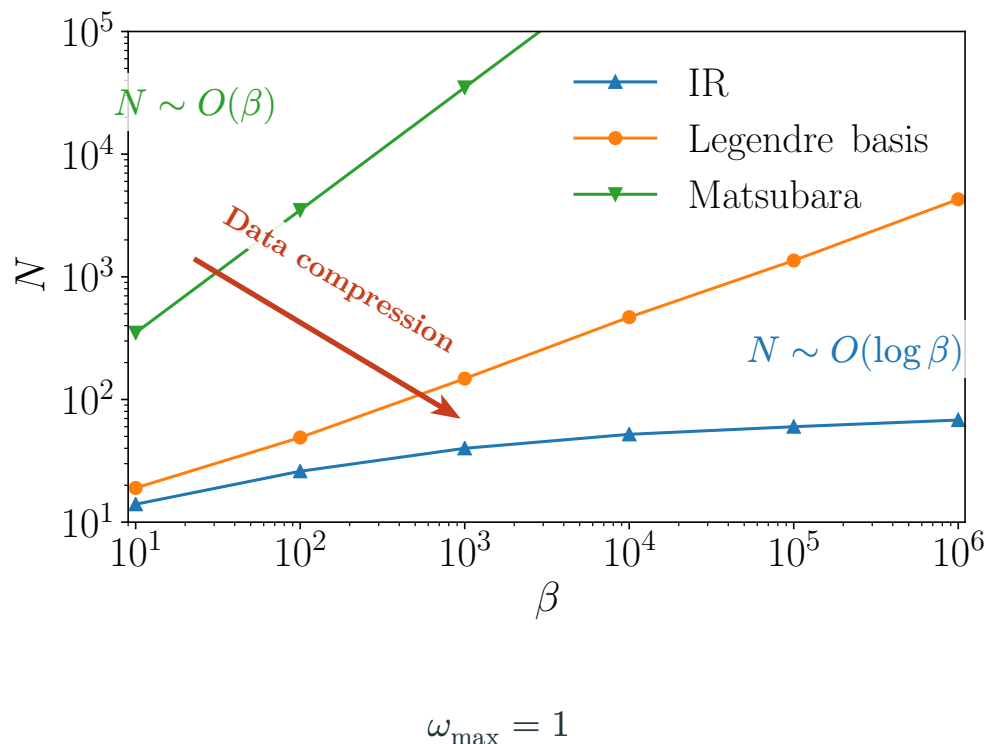
Each insight opened a line of work, now pursued by many groups.

Earlier roots: kernel SVD in analytic continuation (1995–); quantics tensor trains in applied mathematics (Oseledets, Khoromskij, 2010–11).

Q1: Can we compress Matsubara frequencies?



Naive Matsubara dense grid $G(i\omega_n)$: spacing $\sim T$ + slow tail $\sim 1/\omega_n^k \rightarrow N \sim O(\beta)$, intractable at low T



$T \sim 1$ K vs. band energies ~ 10 eV:
 $\beta\omega_{\max} \geq O(10^4)$

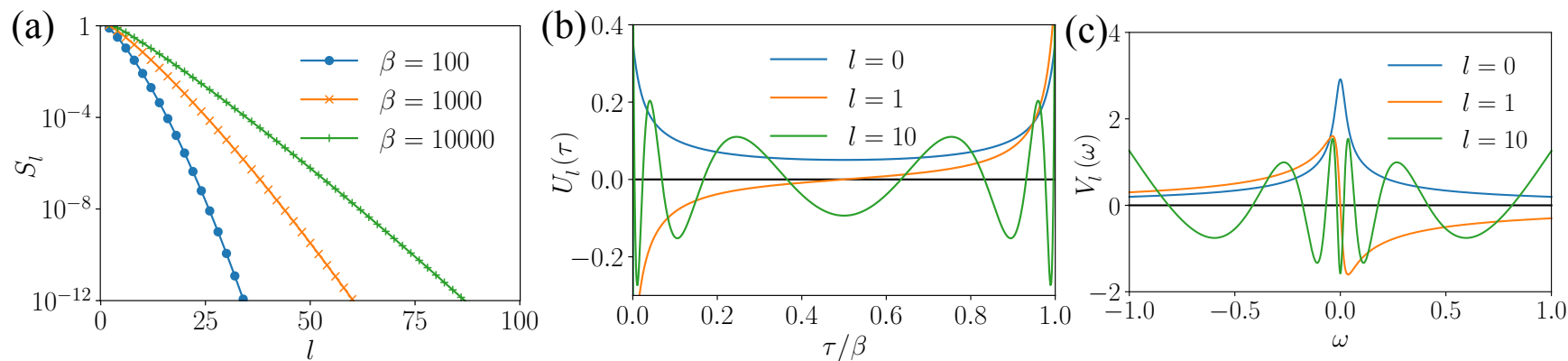
sparse-ir¹: library for many-body methods

- Python / Julia / Fortran / C++ / Rust
- *ab initio* GW, Eliashberg, DMFT, vertex

¹HS, Otsuki, Ohzeki, Yoshimi, PRB **96**, 035147 (2017); Li, Wallerberger, Chikano, Yeh, Gull, HS, PRB **101**, 035144 (2020)

$$G(\tau) = - \int_{-\omega_{\max}}^{\omega_{\max}} d\omega K(\tau, \omega) \rho(\omega), \quad K(\tau, \omega) = \sum_l U_l(\tau) S_l V_l(\omega)^1$$

- $\rho(\omega)$: system-dependent spectrum, K : system-independent kernel
- S_l : singular values, $U_l(\tau)$ and $V_l(\omega)$: singular functions (**IR basis**)



Coefficients $G_l = -S_l \int d\omega V_l(\omega) \rho(\omega)$ decay as rapidly as S_l .

¹HS, Otsuki, Ohzeki, Yoshimi, PRB **96**, 035147 (2017); Otsuki, Ohzeki, HS, Yoshimi, JPSJ **89**, 012001 (2020)

- Sparse τ and $i\omega_n$ grids from the IR basis: $N \propto \log(\omega_{\max}\beta)$
- Interpolate G on the sparse grids via the IR basis
→ FFT, convolution, and elementwise operations are efficient

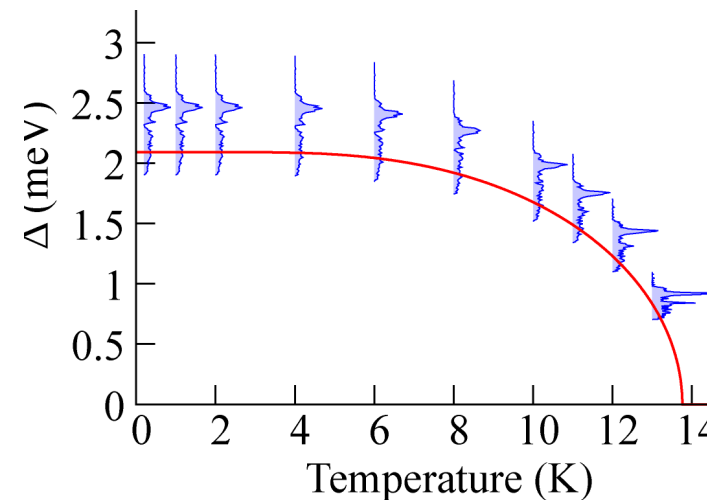


- **Anisotropic Migdal-Eliashberg theory** with Wannier interpolation, using the IR basis

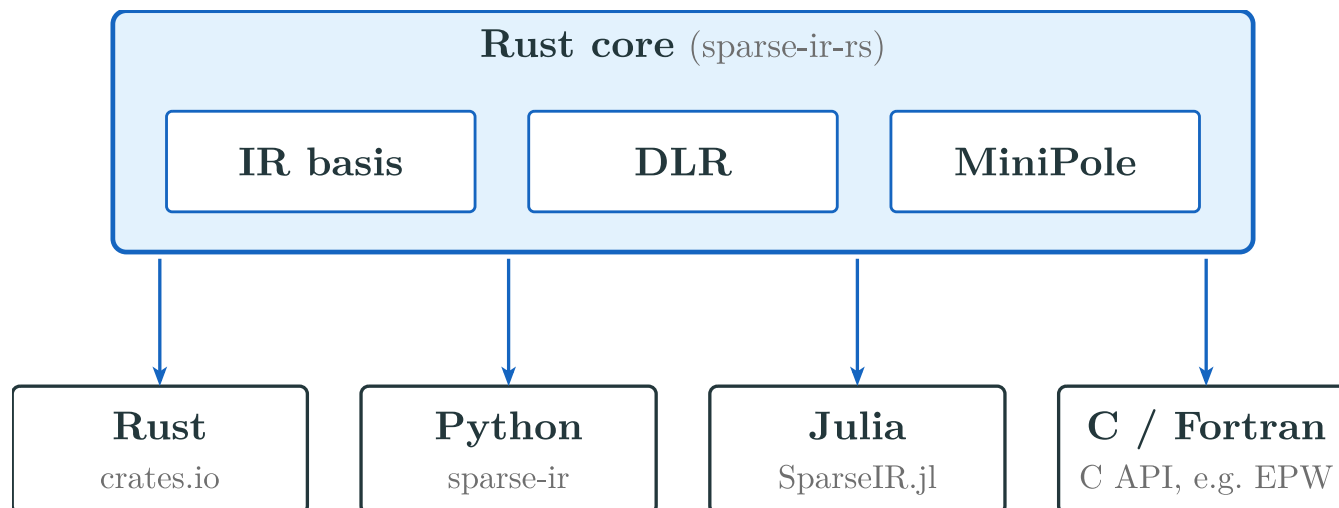
Mori, Nomoto, Arita, Margine, PRB **110**, 064505 (2024)

- Applied to Al, Nb, Pb, and hydrides
- Implemented in the EPW package
- *GW*-Eliashberg benchmark on the uniform electron gas

Akashi, **HS**, PRB **113**, 014507 (2026)



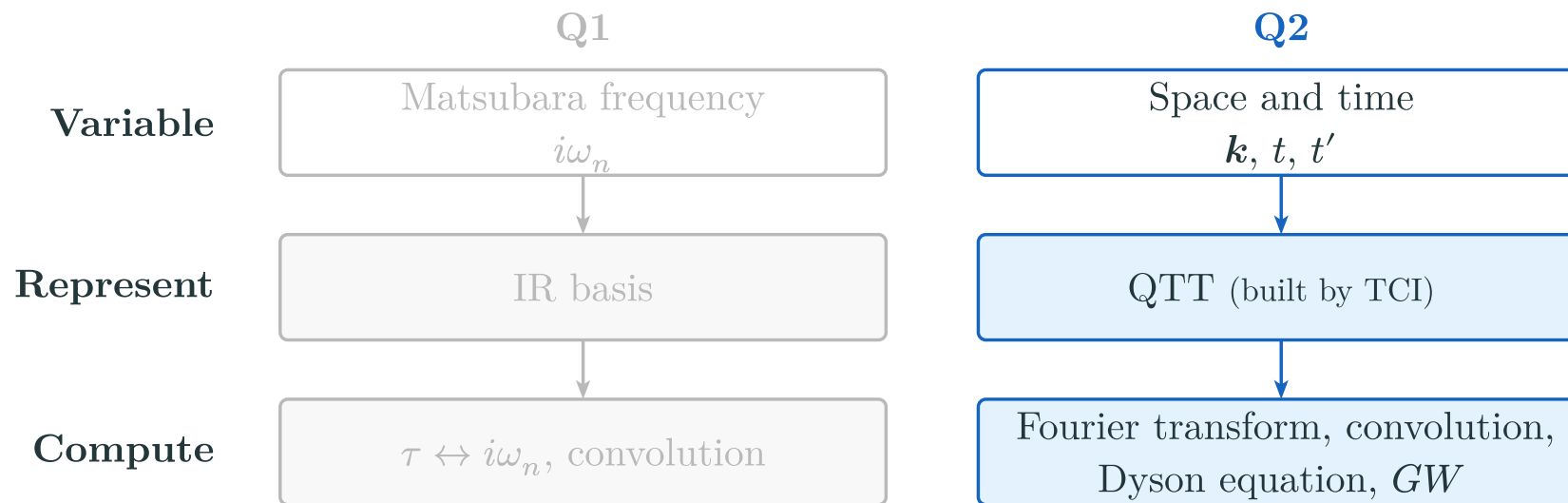
Mori+ 2024: $\Delta(T)$ for Nb
 $O(\log \beta)$ scaling $\rightarrow$ reach $O(1)$ K



- **IR basis**: compress $G(\tau)$, $G(i\omega_n)$; transform $\tau \leftrightarrow i\omega_n$ from a few sampling points
- **DLR**: G as a sum of a few real poles; convolutions and Dyson equation in τ
- **MiniPole**: analytic continuation to real frequencies with a few complex poles

spm-lab.github.io

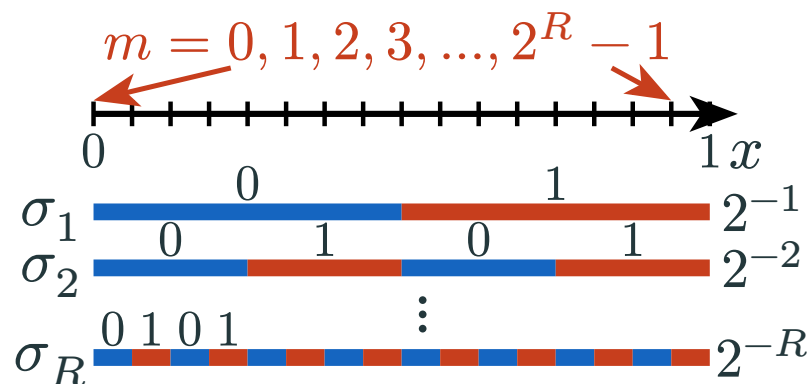
Q2: What about space and time?



First systematically formulated in applied math¹

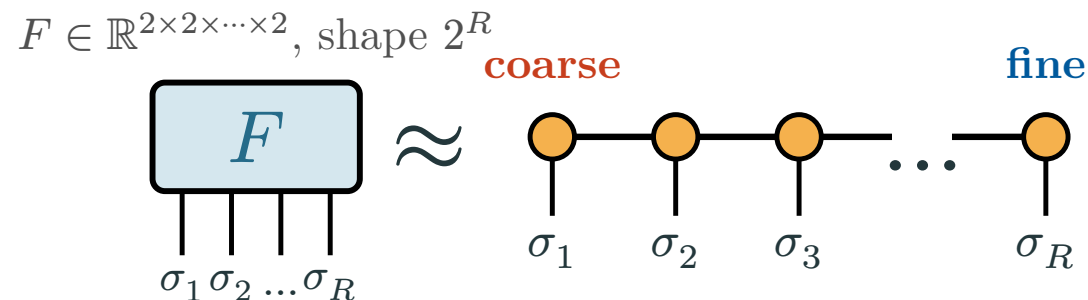
Tensorize the grid

- Discretize $f(x)$ on $N = 2^R$ grid
 - Write the index in binary: $m = (\sigma_1 \sigma_2 \dots \sigma_R)_2$
- Reshape 2^R values $\rightarrow R$ -way tensor F



Use scale separation

- If scales separate, F factorizes as a tensor train
- Small bond $\chi \rightarrow O(R\chi^2)$ parameters and exponentially fine scale resolution with R .



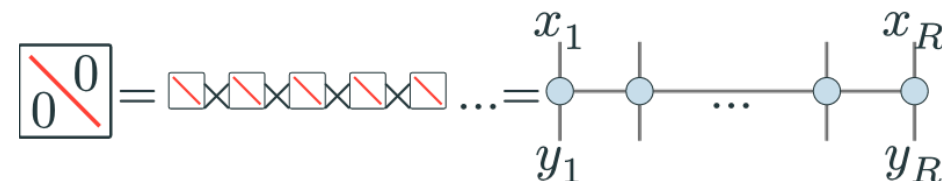
¹Oseledets, Dokl. Math. **80**, 653 (2009); Khoromskij, Constr. Approx. **34**, 257 (2011).

Provably compressible ($\chi = 1$ or $O(1)$):

- $f(x) = e^x = e^{x_1/2} e^{x_2/4} \dots e^{x_R/2^R} \rightarrow \chi = 1$
 - Sum of N exponentials $\rightarrow \chi \leq N$
- Polynomial of degree $p \rightarrow \chi \leq p + 1$
- Heaviside step function $\rightarrow \chi = 1$
- $f(x, y) = \delta_{x, y} = \delta_{x_1, y_1} \delta_{x_2, y_2} \dots \delta_{x_R, y_R} \rightarrow \chi = 1$

Empirically compressible:

- Extended features + localized features



Identity matrix: $\chi = 1$ Matrix Product Operator (MPO)

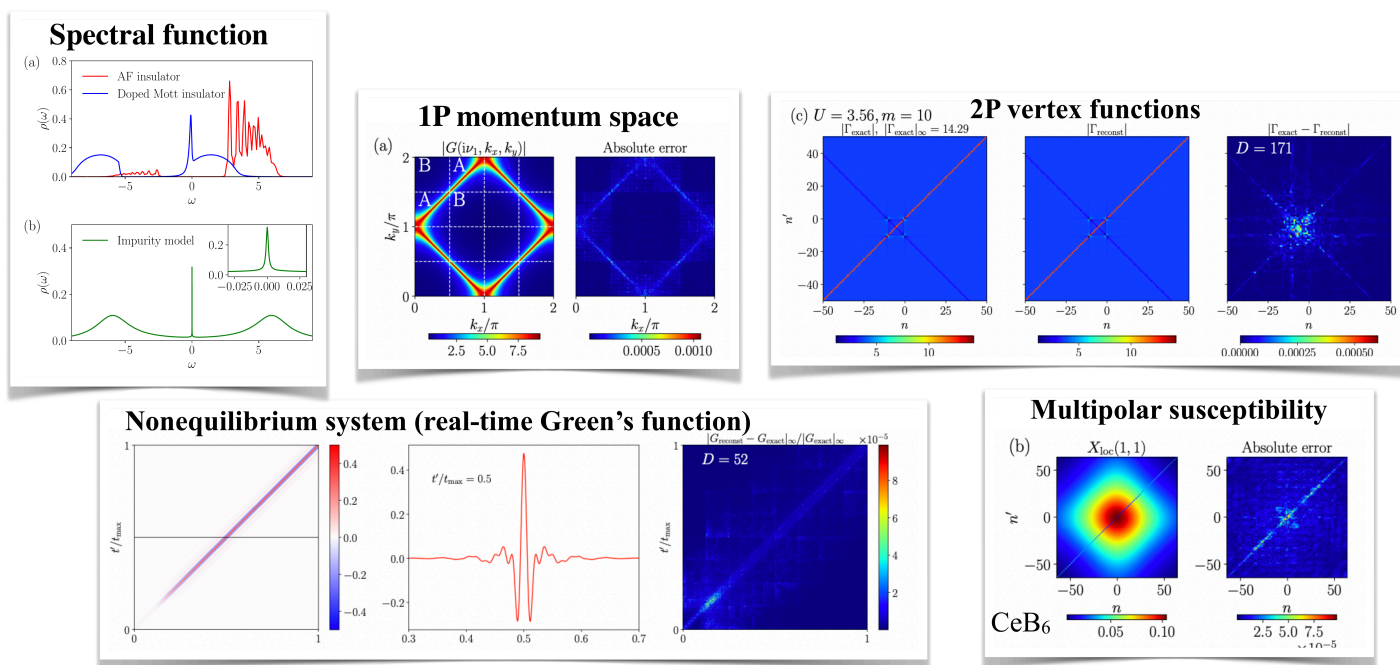
TensorNetwork.org: Representation of Continuous Functions

Khoromskij, Tensor Numerical Methods in Scientific Computing (2018)

Khoromskij, Constr. Approx. **34**, 257 (2011); HS+, Wallerberger, Murakami, Nogaki, Sakurai,

Werner, Kauch, PRX **13**, 021015 (2023)

- Frequency, momentum, and real time are compressible.
- Fundamental diagrammatic operations are efficient in the compressed form.

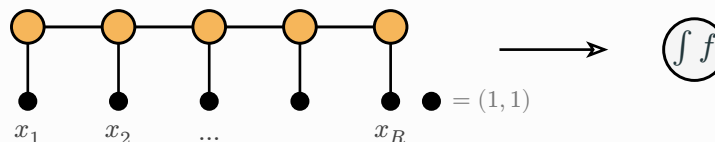


HS, Wallerberger, Murakami, Nogaki, Sakurai, Werner, Kauch, PRX **13**, 021015 (2023)

Computational costs scale linearly with R .

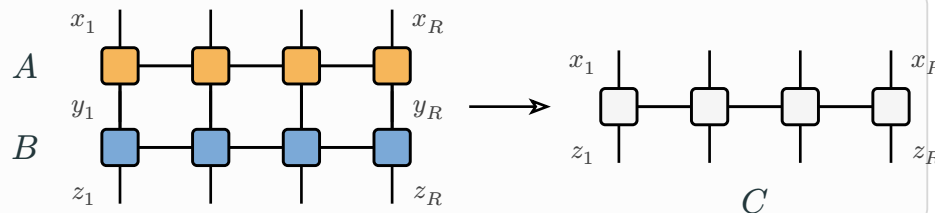
Integration

$$\sum_{\mathbf{x}} f(\mathbf{x})$$



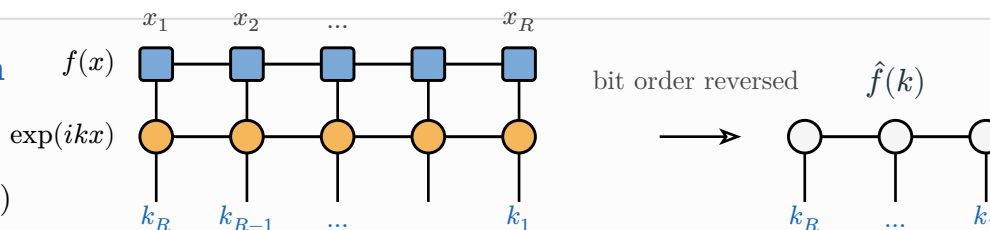
Convolution

$$\sum_y A(x, y) B(y, z) \approx C(x, z)$$



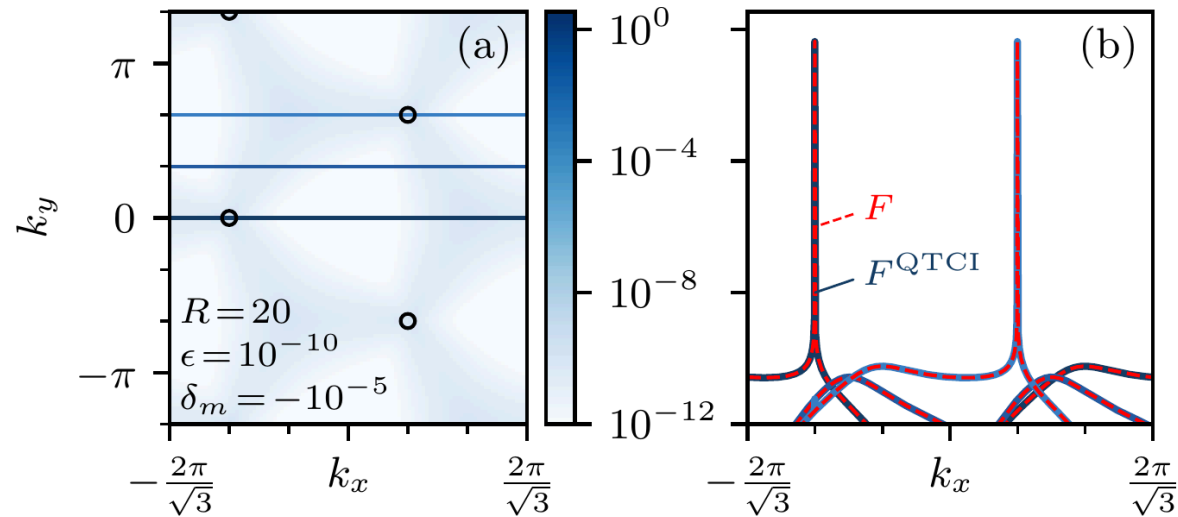
Fourier transform

$$\hat{f}(k) = \sum_x \exp(ikx) f(x)$$

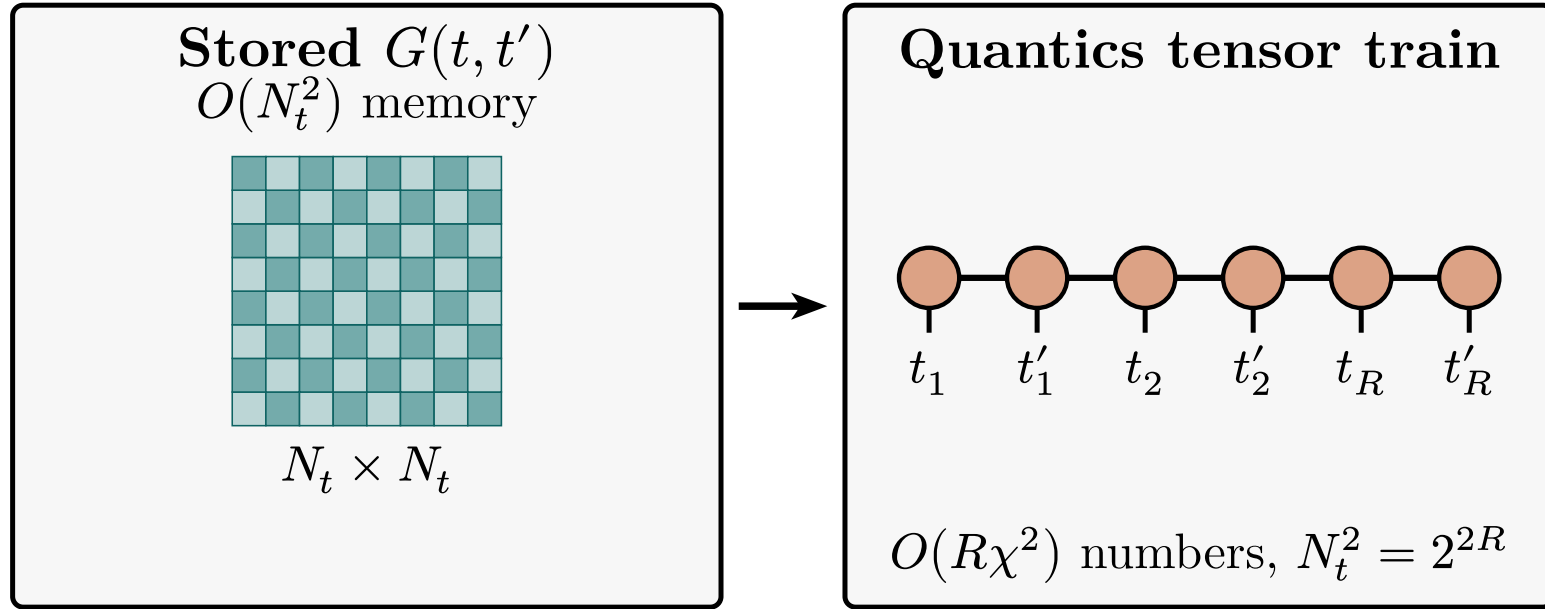


- A typical MPO bond dimension for Fourier transform is $\chi \approx 15$ for machine precision.
- PDE/ODE can be solved in the compressed form.

- Learns a TT from a **black-box** f by adaptive queries
- $O(\chi^2 L)$ function calls; never reads the full tensor
- Used for $\mathbf{k}$ sums and for real-time Green's functions



Brillouin-zone integral on 2^{3R} points from few samples



- Nonequilibrium dynamics needs $G(t, t')$: two time variables
- Dyson equation solved in compressed (QTT) form

Murray, **HS**, Werner, PRB **109**, 165135 (2024)



M. Środa

U. Fribourg

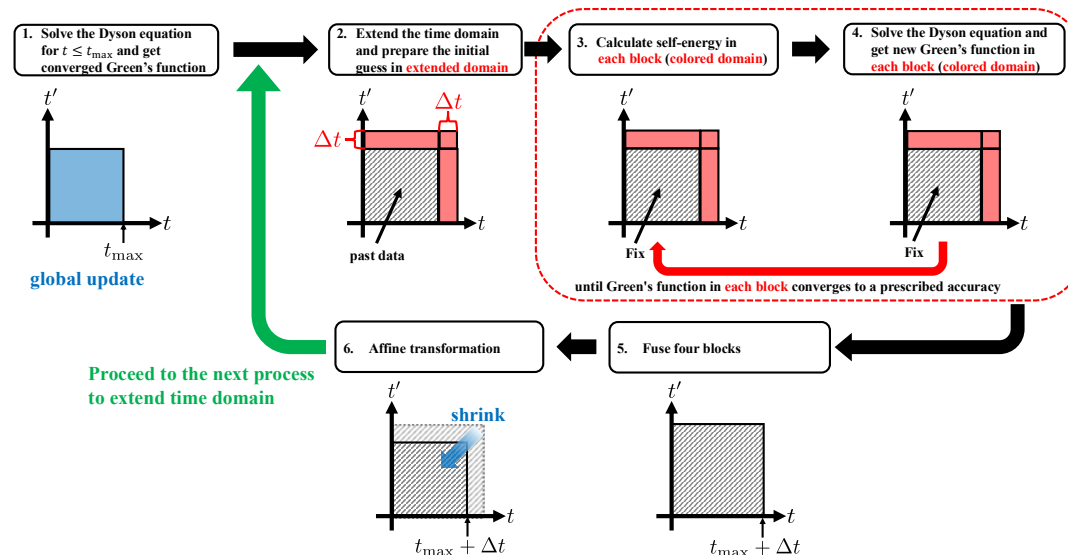
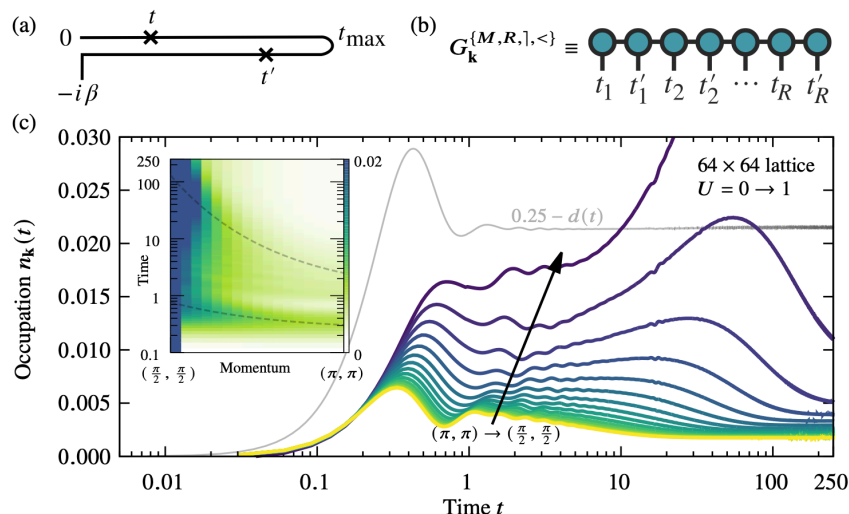
Compress $G(t, t')$ in QTT form



K. Inayoshi

Saitama U.

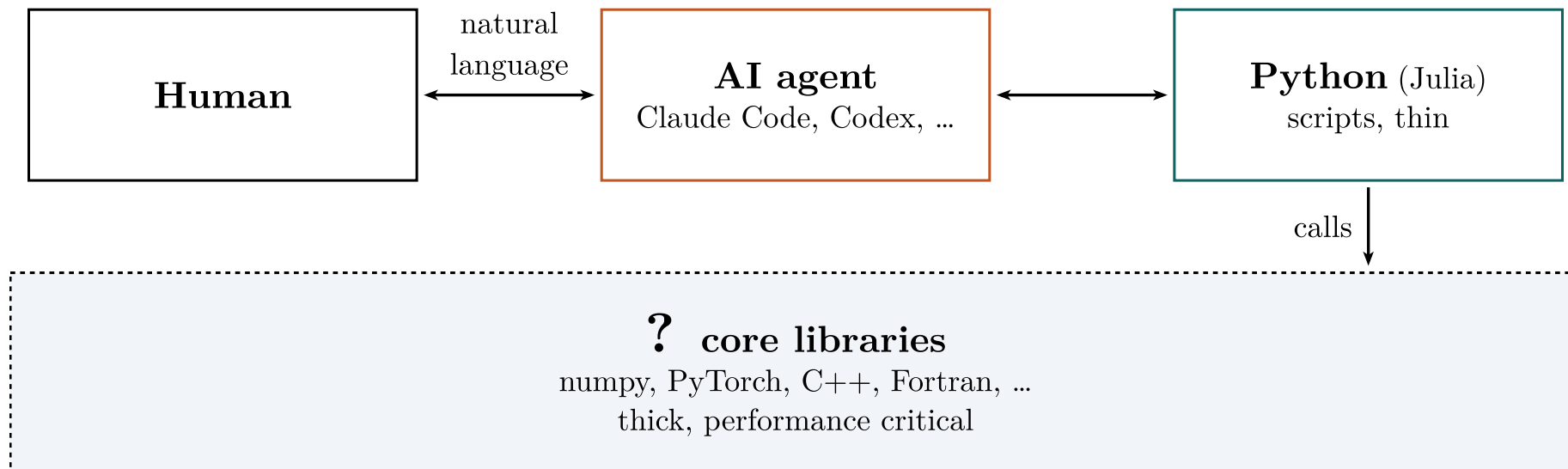
Reach long times: causal divide-and-conquer



- **Memory: 6 TB $\rightarrow$ 1 GB**, 64×64 k grid
- Dyson and GW solved in QTT form

- Propagate forward in time using **causality**
- Extend $t_{\max}$ in pieces, reuse the past

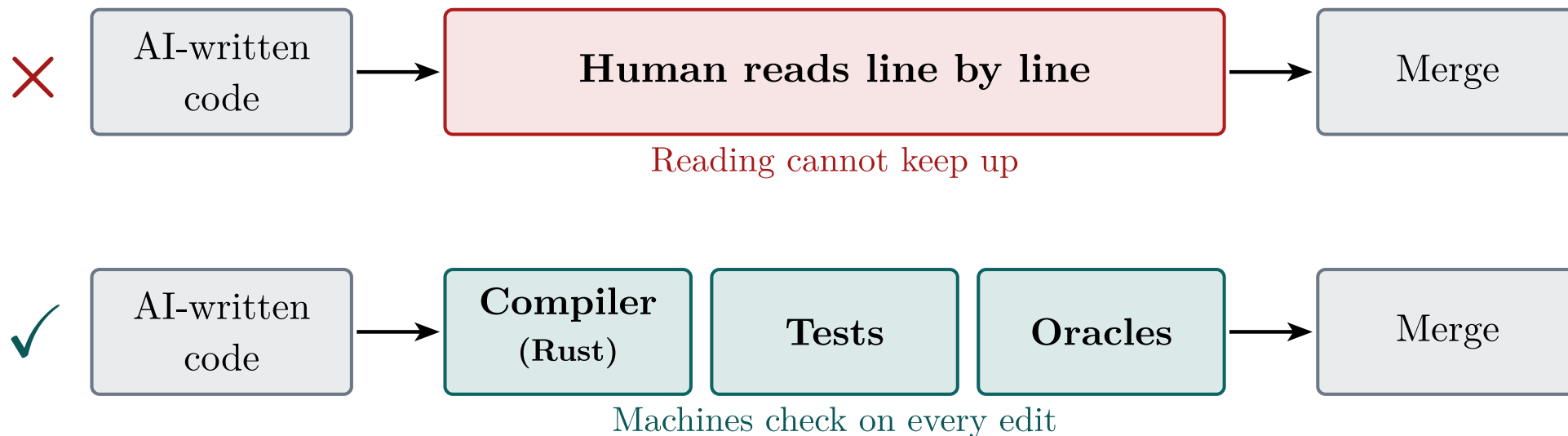
Part II: Reliable software with AI agents



- Analysis scripts (Python / Julia) on top are thin
- Performance is determined by the **core libraries** below

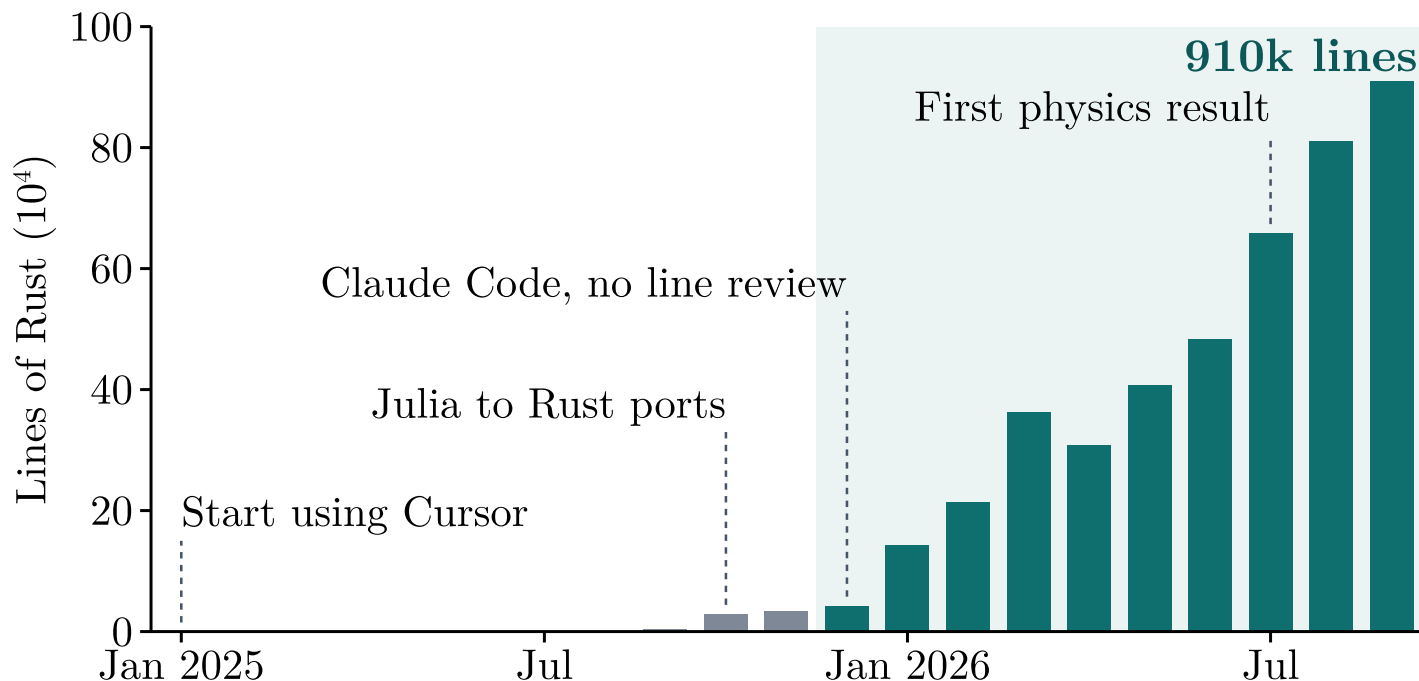
Question: which **language** for the core libraries?

- “AI slop”: curl ended its bug bounty after a surge of AI-generated reports (2026)



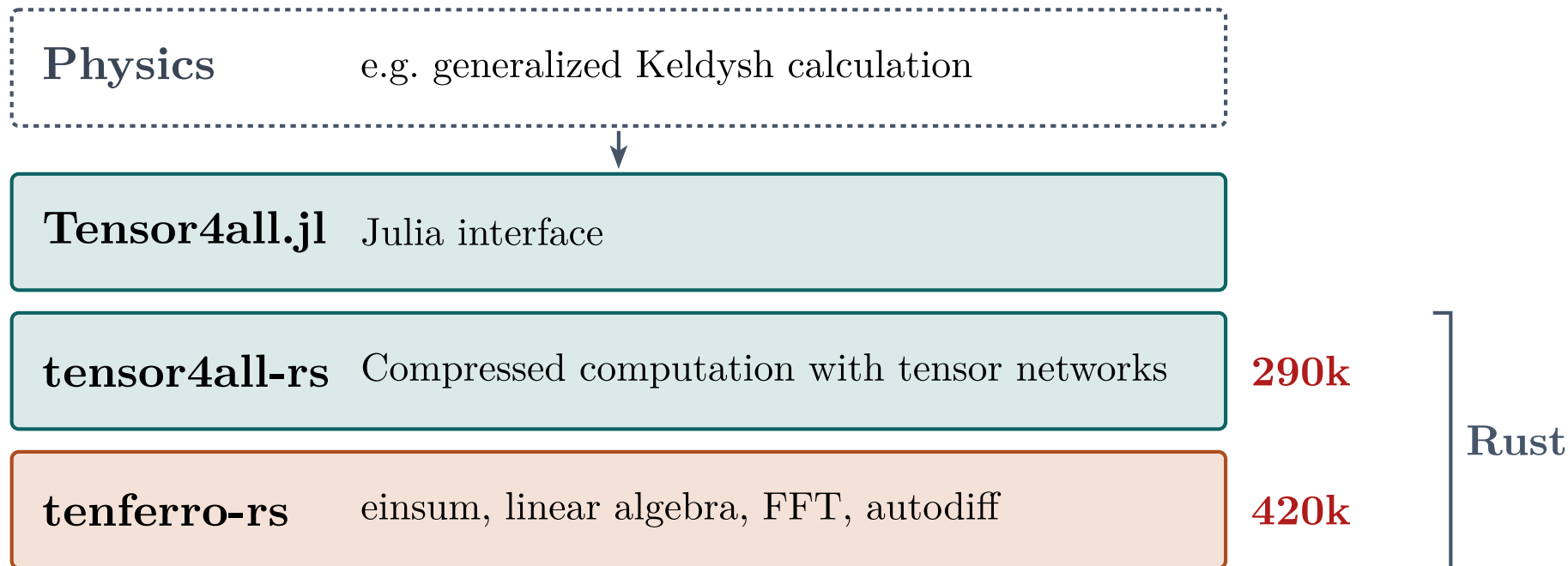
Move correctness checks from **reading code** to **machine verification**

The Register (2026-01-21): curl ends bug bounty



- 910k lines of Rust, none written by hand since January 2026
- Up to 108,110 lines in one week: **line-by-line review cannot keep up**

Rust lines on default branches of 15 research repositories, including tests and benchmarks (2026-09-23). Weekly maximum: 2026-W13, 6 main repositories.

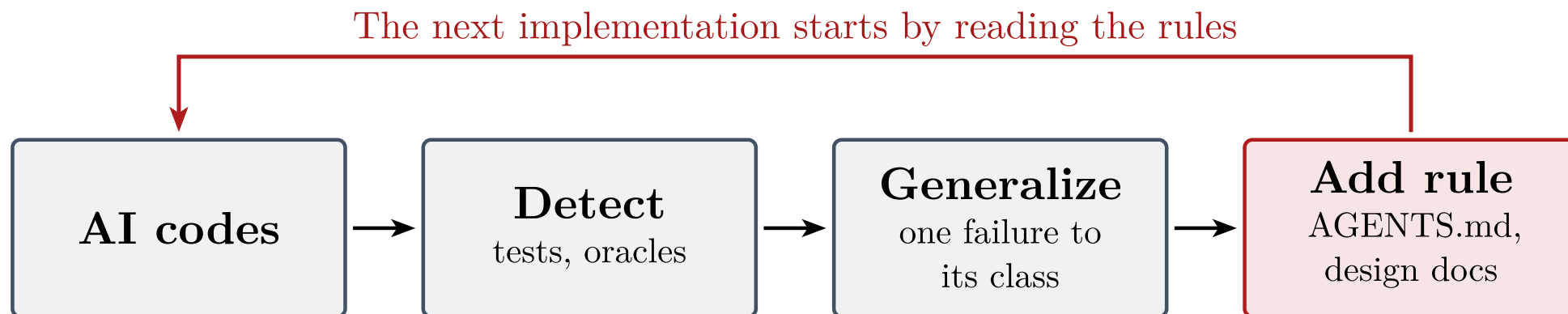


- Lower layers are more general; tenferro-rs is also used by 9 external projects
- First physics result: by a researcher **new to Rust**, in about two months

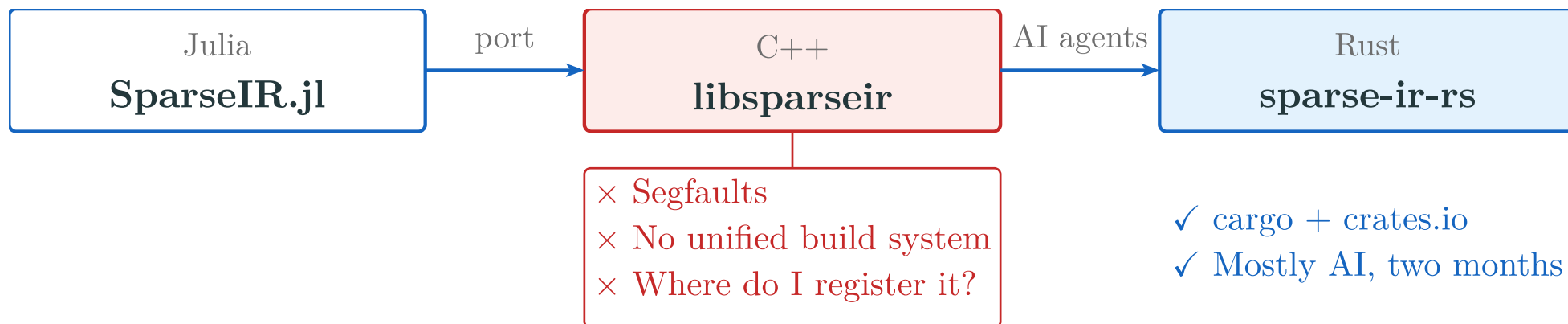
Line counts include tests (2026-09-23). K. Inayoshi, H. Shinaoka, T. Murakami, arXiv:2607.11055

Loosen tolerance $10^{-12} \rightarrow 10^{-3}$	Hand-roll matmul Instead of BLAS	Add a data race Parallel sum, shared var
Compiles ✓	Compiles ✓	Compiles ✓
Tests pass ✓	Tests pass ✓	Tests pass ✓
Accuracy drops ✗	Orders slower ✗	Varies per run ✗

In every case the code **compiles and the tests pass**



- Example: new rule “Do not loosen tolerances; report the cause”
- The same kind of failure becomes **less likely** in the next implementation

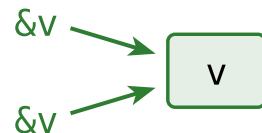


“I started to hate C++ (again).”

zenn.dev/h_shinaoka/articles/a099b814166ac7 (Why I migrated from C++ to Rust)

	C++ / Fortran	Julia	Rust
No segfault in long runs	✗	✓	✓
No silently wrong results (aliasing)	✗	✗	✓
Efficient parallelism without GC	✓	✗	✓
Unified package manager and build	✗	✓	✓
Callable from C, no runtime	✓	✗	✓

✓ **shared reads: any number**



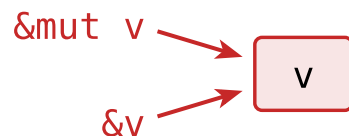
read-only references

✓ **exclusive write: exactly one**



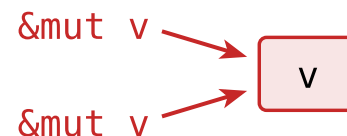
one writable reference

✗ **read while writing**



an alias would observe the write

✗ **two writers**



two write permissions at once

Shared reads or **one exclusive write**. Enforced at compile time

C++ / OpenMP

```
#pragma omp parallel for  
for (int i = 0; i < n; ++i)  
    total += x[i];
```

Compiles; wrong sum at run time (data race)

Rust / rayon

```
x.par_iter()  
    .for_each(|xi| total += xi);
```

```
error[E0596]: cannot borrow  
`total` as mutable
```

Rust stops at compile time. The correct form is `x.par_iter().sum()`.

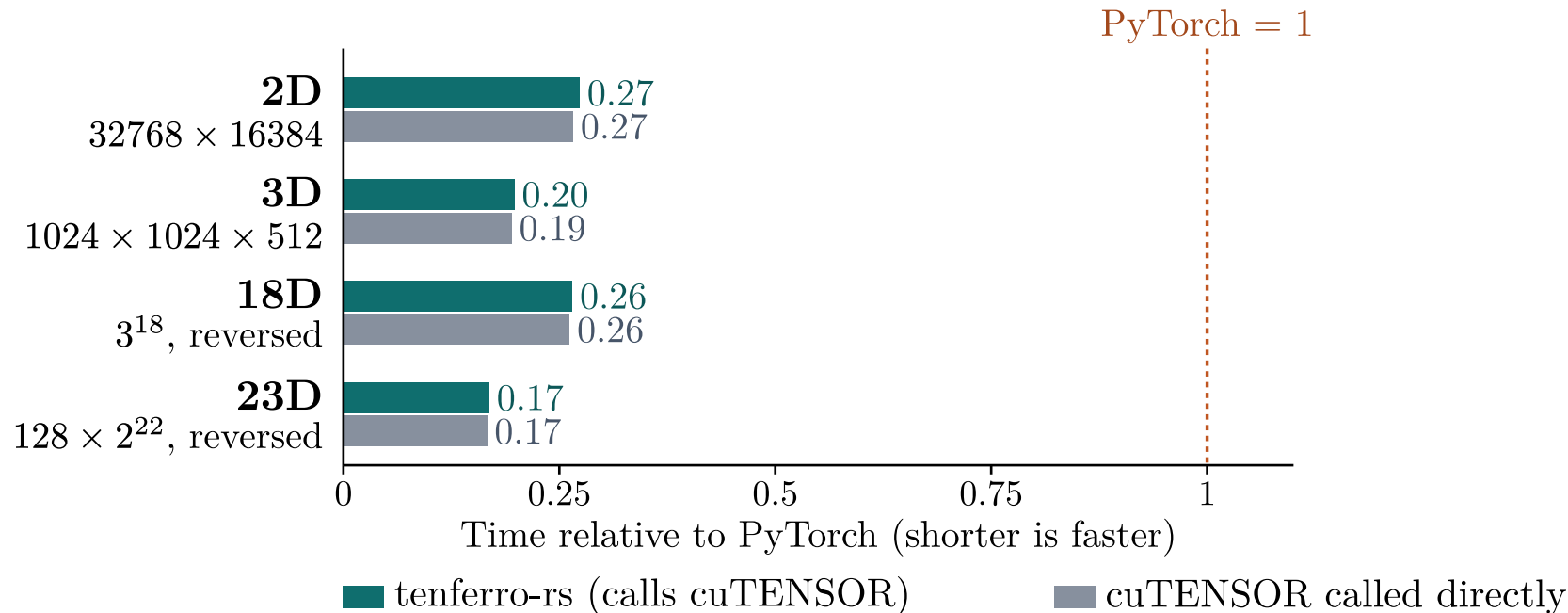
March 2023: “Julia is easier. Rust has a steep learning curve and few numerical libraries.”

Obstacle in 2023

In 2026

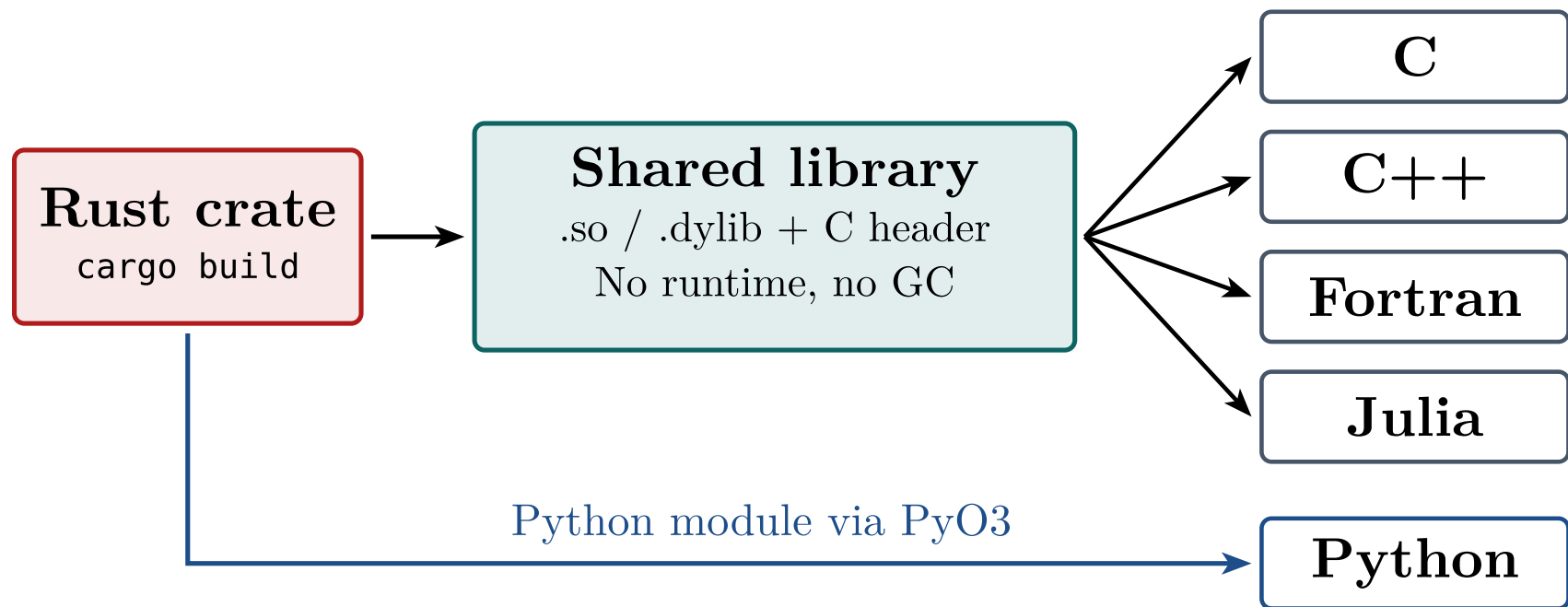
- | | | |
|-------------------------|---|--|
| Steep learning curve | → | AI writes the code |
| No GPU vendor support | → | CUDA Rust (NVIDIA, 2026-09-08) |
| Few numerical libraries | → | Agents port them from other languages |

Priority shifts from ease of writing to **ease of checking**

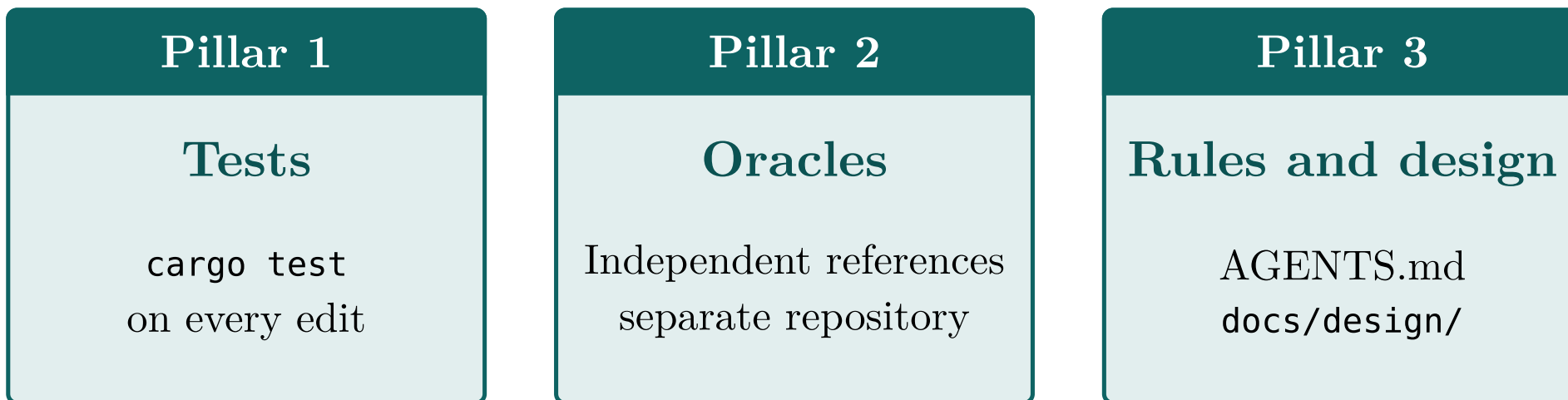


- Calls cuTENSOR internally: **3.7 to 5.9 times faster** than PyTorch
- Dense operations (matmul, eigendecomposition): on par with PyTorch

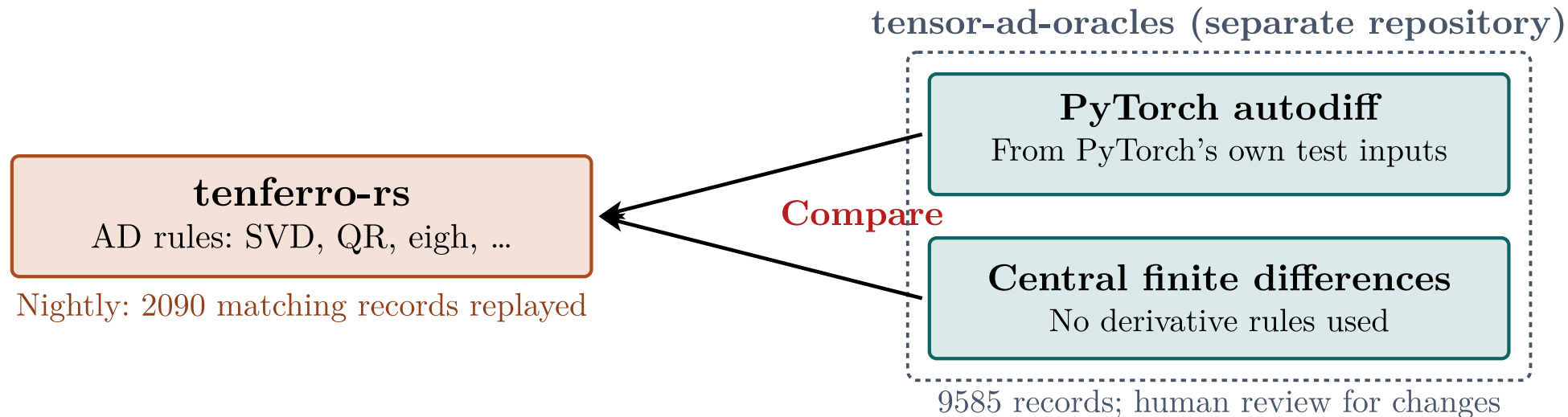
tenferro-benchmark (2026-09-17, A100, f64, 4 GiB each, output buffer reused)



- **C ABI** for Fortran / C++ callers (sparse-ir-rs from EPW)
- `cargo vendor` builds on machines without network access

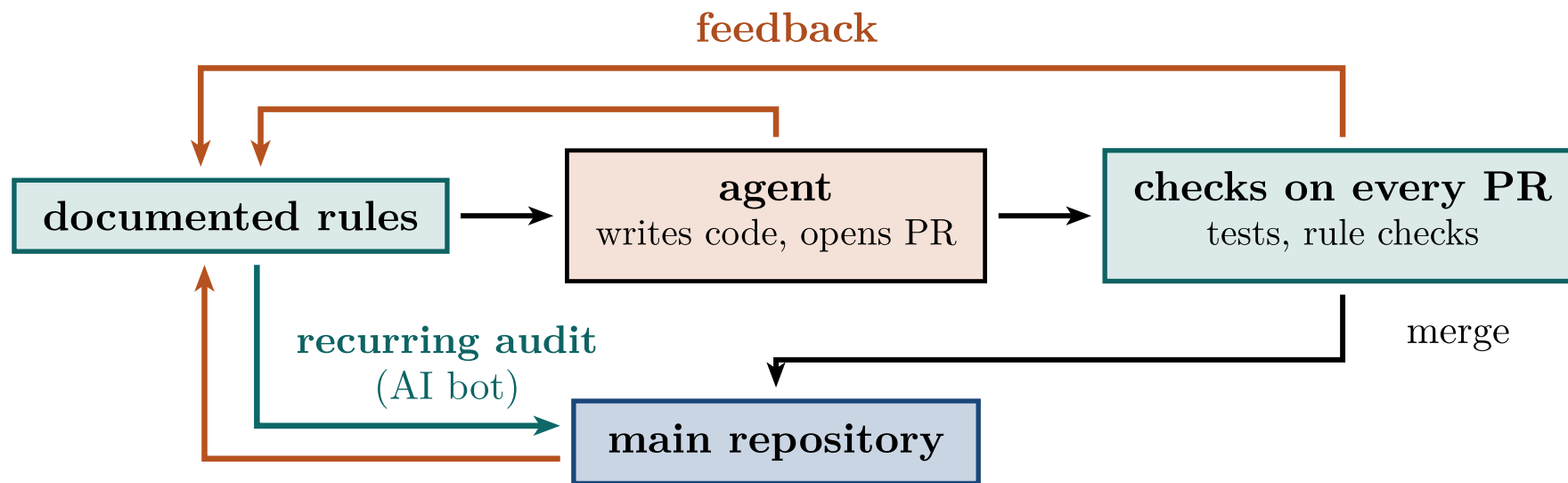


Instead of reading every line, **verify with these three**

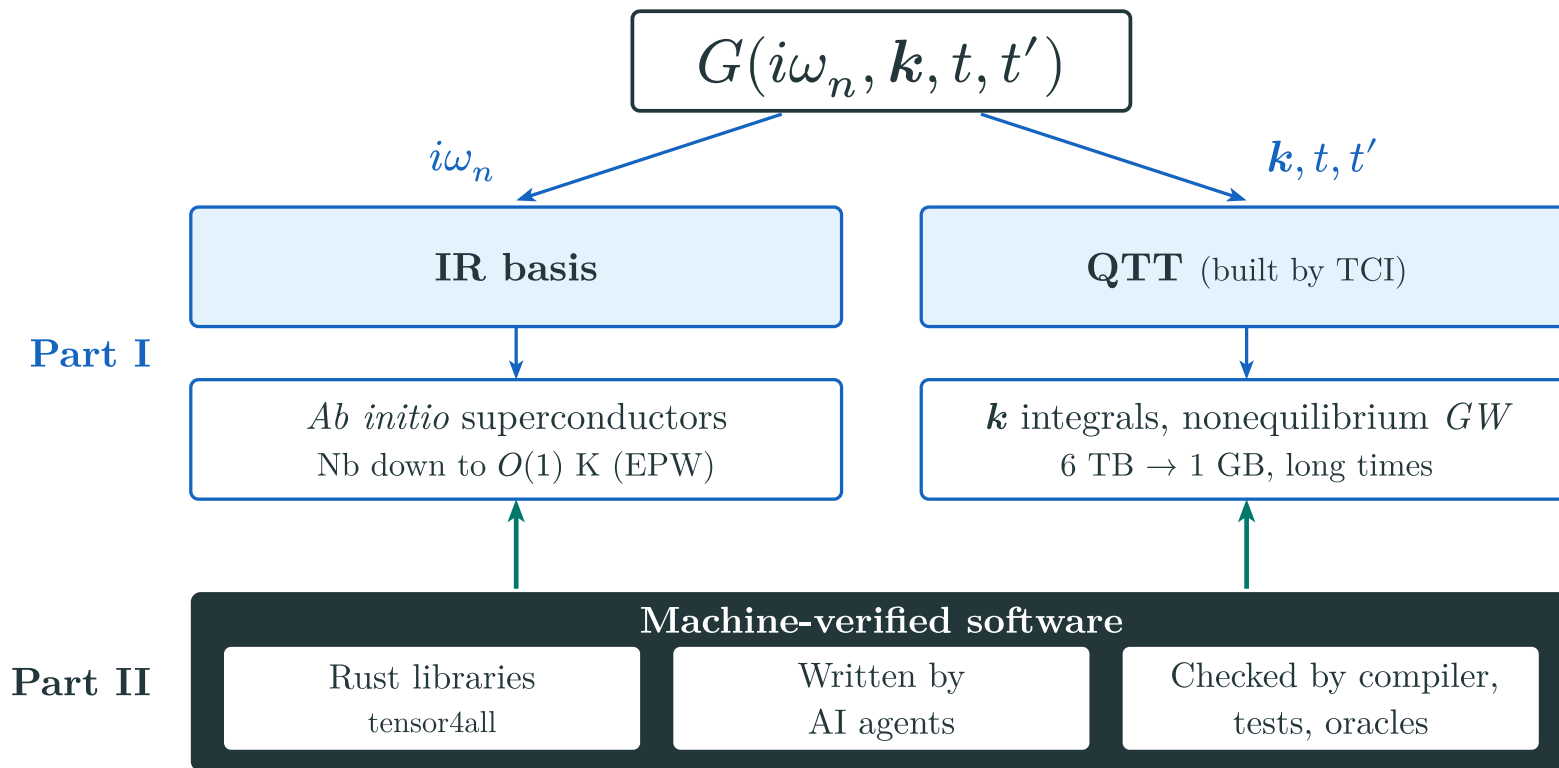


- Derivative rules are checked against **two independently built references**
- References live in a separate repository that agents cannot modify

github.com/tensor4all/tensor-ad-oracles



- A bot checks every PR against the documented rules
- Oracles and benchmarks are in **repositories that agents cannot touch**



Slides (PDF)

shinaoka.github.io · tensor4all.org · spm-lab.github.io